Design Recommendation & Guidelines for Domain-Driven Design (DDD)

Transitioning existing API-Led integrations into a Domain-Driven Design (DDD) architecture requires shifting the organizational focus away from the underlying technical systems (like SAP, Salesforce, or TrackWise) and geographic regions (NA vs. EMEA), and instead organizing around core business capabilities (e.g., Order, Shipment, Person, Case).

While the fundamental API-Led layers (Experience, Process, System) remain, DDD changes how you draw the boundaries around those layers.

Here are the design recommendations for moving your existing APIs to DDD, followed by the steps to build future APIs from scratch based on the modernization strategy.

Moving Existing API-Led APIs to DDD

When refactoring your current inventory, apply the following integration policies:

1. **Restructure by Domain, Not by System**: Existing APIs are currently heavily fractured by the systems they connect to (e.g., dex-na-orders-sys-api, sap-account-order-sys-api). You must refactor and group these APIs strictly by their business boundaries (e.g., all order logic belongs to a single "Order Domain" API). Never mix business logic across different domains (e.g., do not mix Order and Case logic in the same flow).
2. **Consolidate Regional APIs via Parameterization**: Instead of maintaining separate North American and European APIs (e.g., merging emea-order-sync-prc-api and na-order-sync-prc-api), consolidate them into a single, global generic API (e.g., order-sync-prc-api). To handle this, the global API must require a region parameter (e.g., NA or EMEA) set from the runtime context.
3. **Abstract System Integrations into Adapters:** Because you are consolidating global APIs, you must implement Region-Specific Adapters inside the APIs. The generic API will use the region parameter to dynamically select the correct adapter at runtime. These adapters act as the translation layer, handling the specific field mapping to the backend system (e.g., Salesforce NA vs. SAP EMEA) and enforcing regional compliance rules.
4. **Implement Single Canonical Models**: Transition away from exposing system-specific payloads. You must define a singular list of canonical fields (one schema) per business entity that supports both NA and EMEA. Fields that only apply to one region (like an EMEA customs reference) should be marked as optional, and highly specific system data should be placed into a single optional extension object.

5. **Shift to Event-Driven Decoupling:** Move away from scheduled batch jobs and point-to-point calls in favor of Event-Driven Architecture (EDA) using Anypoint MQ and Salesforce Platform Events for real-time synchronization.
6. **Resolve Layering Anti-Patterns:** Ensure your existing Experience APIs are only orchestrating business logic via Process APIs. It is flagged as an architectural anti-pattern for an Experience API to bypass the Process layer and call a backend System API (like SAP) directly.

Steps for Setting Up Future DDD APIs

When designing a new API from scratch, follow this sequence to ensure it complies with the DDD strategy:

Step 1: Define the Business Domain and Naming Convention

- Identify the exact business capability the API serves (e.g., Shipment, Document, Product).
- Use noun-based, domain-centric resource naming for REST endpoints (e.g., use /orders/international instead of verb/system-heavy names like /intl-orders/initial). Remove all regional identifiers (na, emea) and system acronyms (sf, sap) from the application name.

Step 2: Design the Global Canonical Schema

- Before writing integration logic, define the canonical model inspired by industry standards, but keep it pragmatic.
- Ensure the schema mandates the region parameter.
- Ensure all dates are strictly ISO 8601 formatted, and IDs are passed as strings.

Step 3: Separate Ingestion Logic from Business Processing

- If the new API is triggered by an event, do not bundle the business orchestration into the listener. Build a dedicated Event Listener API (e.g., order-event-listener) whose sole responsibility is to receive the event, validate the payload against the canonical schema, and publish it to Anypoint MQ.
- A separate Process API should then subscribe to that MQ topic to execute the actual business workflow.

Step 4: Enforce Centralized Cross-Cutting Utilities

- **Idempotency**: Because the architecture relies heavily on MQ and event replays, implement strict idempotency checks (using hashes, record IDs, or timestamps) to prevent duplicate record processing.

- **Logging & Error Handling**: Do not build custom error handling. Integrate the shared common-logging-error-handling-module to ensure structured logging (JSON/key-value pairs), standard FHIR/HL7 error models (AuditEvent/OperationOutcome), and centralized PHI/PII redaction